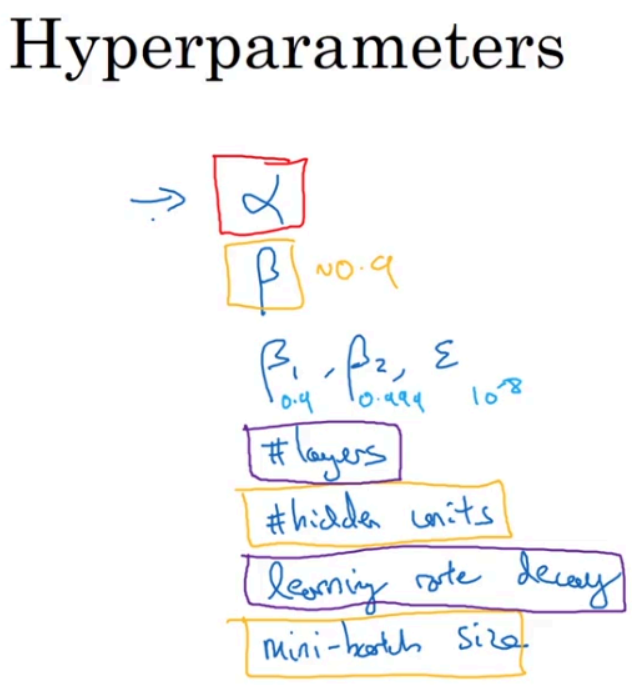
Hyperparameter Tuning

Tuning Process

In this video, we will learn some tips for how to systematically organize our hyperparameter tuning process, which hopefully will make it more efficient for you to converge on a good setting of the hyperparameters.



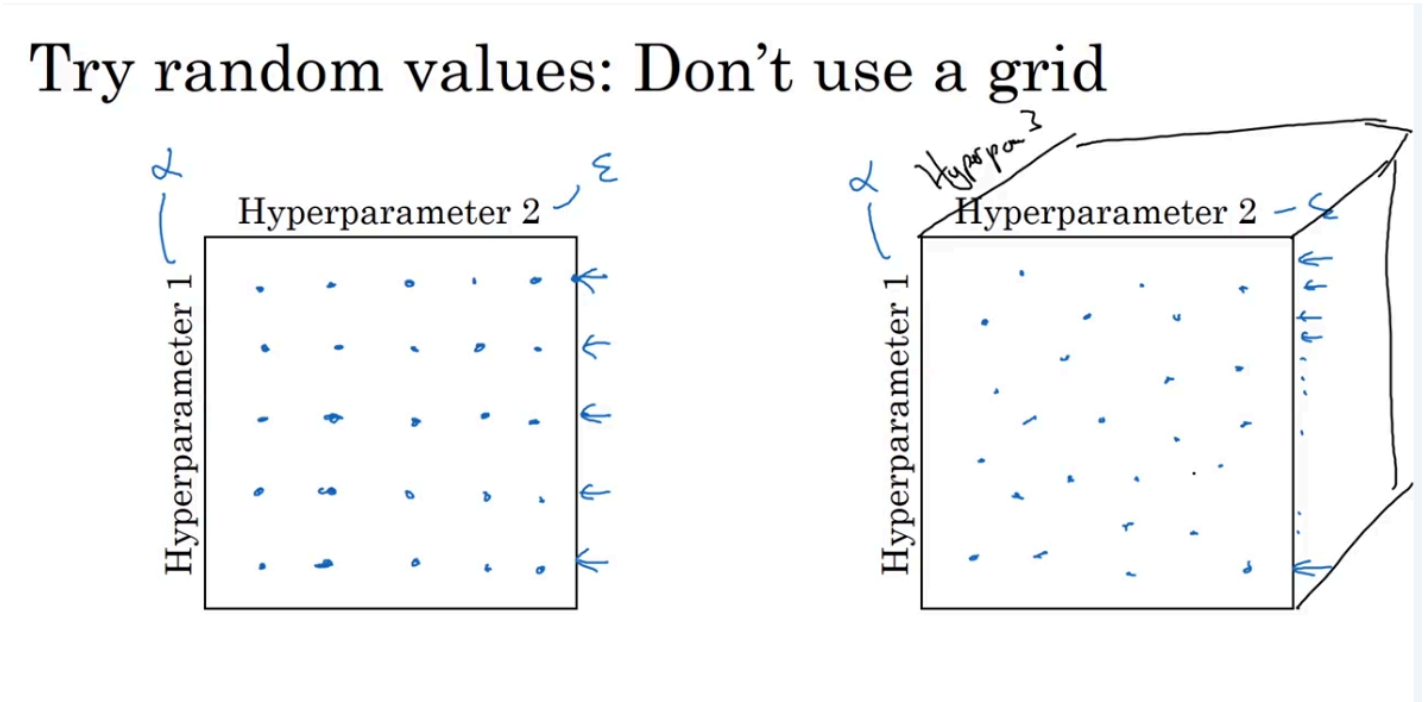
There are a lot of parameters for you to tune, such as:

- learning rate
- momentum (beta)
- hyperparameter for Adam optimization algorithm beta1 beta2 and epsilon
- number of layers
- number of hidden units for each layer
- learning rate decay
- mini-batch size

Most important hyperparameter to tune is learning rate

We could rank them like:

1. Learning rate
2. momentum beta (default should be 0.9), mini-batch size, hidden units
3. learning rate decay, number of layers
4. hyperparameter for Adam optimization algorithm beta1 (0.9), beta2 (0.999) and epsilon(10^-8)



Now, if you're trying to tune some set of hyperparameters, how do you select a set of values to explore? In earlier generations of machine learning algorithms, if you had two hyperparameters, which I'm calling hyperparameter one and hyperparameter two here, it was common practice to sample the points in a grid like so, and systematically explore these values. Here I am placing down a five by five grid. In practice, it could be more or less than the five by five grid but you try out in this example all 25 points, and then pick whichever hyperparameter works best.

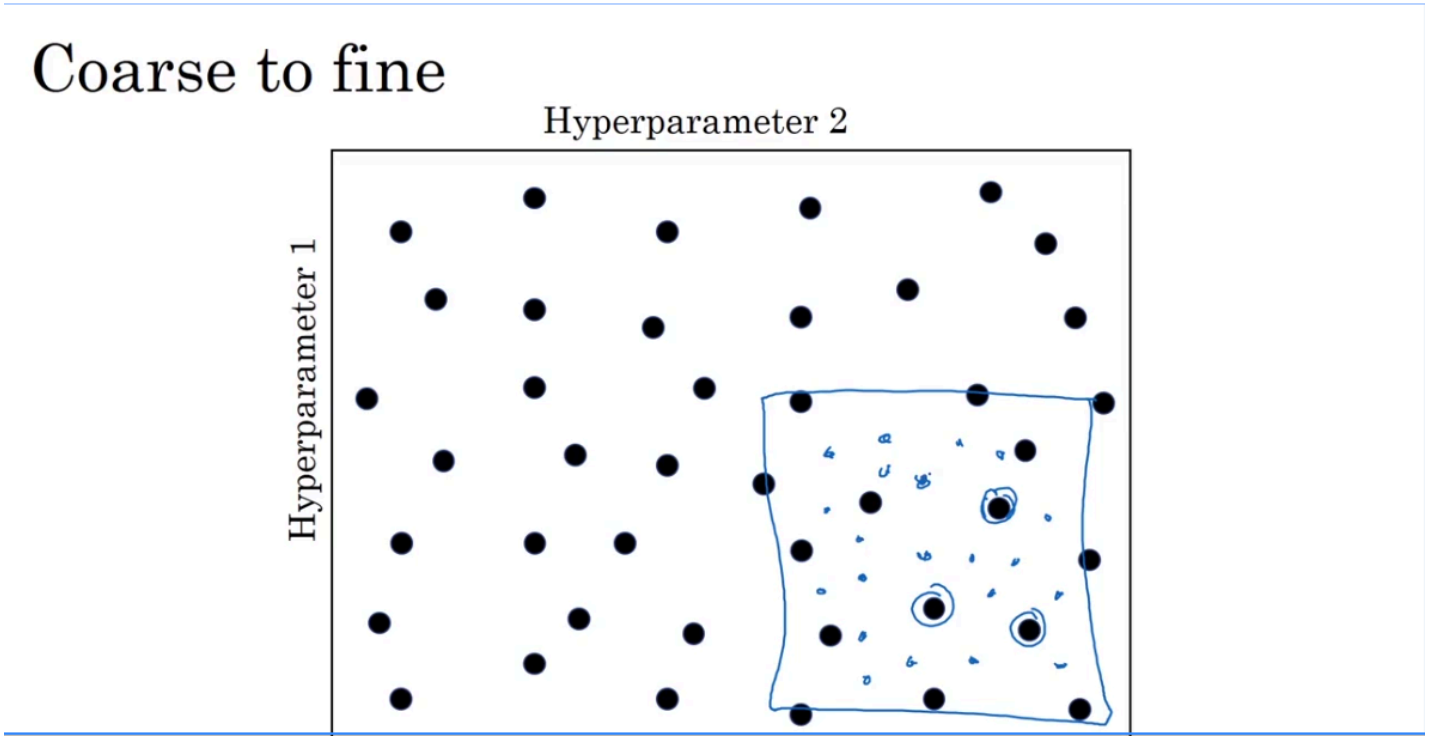
And this practice works okay when the number of hyperparameters was relatively small. In deep learning, what we tend to do, and what I recommend you do instead, is choose the points at random. So go ahead and choose maybe of same number of points, right? 25 points, and then try out the hyperparameters on this randomly chosen set of points. And the reason you do that is that it's difficult to know in advance which hyperparameters are going to be the most important for your problem. And as you saw in the previous slide, some hyperparameters are actually much more important than others. So to take an example, let's say hyperparameter one turns out to be alpha, the learning rate.

And to take an extreme example, let's say that hyperparameter two was that value epsilon that you have in the denominator of the Adam algorithm. So your choice of alpha matters a lot and your choice of epsilon hardly matters. So if you sample in the grid then you've really tried out five values of alpha and you might find that all of the different values of epsilon give you essentially the same answer.

So you've now trained 25 models and only got into trial five values for the learning rate alpha, which I think is really important. Whereas in contrast, if you were to sample at random, then you will have tried out 25 distinct values of the learning rate alpha and therefore you be more likely to find a value that works really well. I've explained this example, using just two hyperparameters. In practice, you might be searching over many more hyperparameters than these, so if you have, say, three hyperparameters, I guess instead of searching over a square, you're searching over a cube where this third dimension is hyperparameter three and then by sampling within this three-dimensional cube you get to try out a lot more values of each of your three hyperparameters.

And in practice you might be searching over even more hyperparameters than three and sometimes it's just hard to know in advance which ones turn out to be the really important hyperparameters for your application and sampling at random rather than in the grid shows that you are more richly exploring set of possible values for the most important hyperparameters, whatever they turn out to be.

→ use array to choose hyperparameter randomly



When you sample hyperparameters, another common practice is to use a coarse to fine sampling scheme.

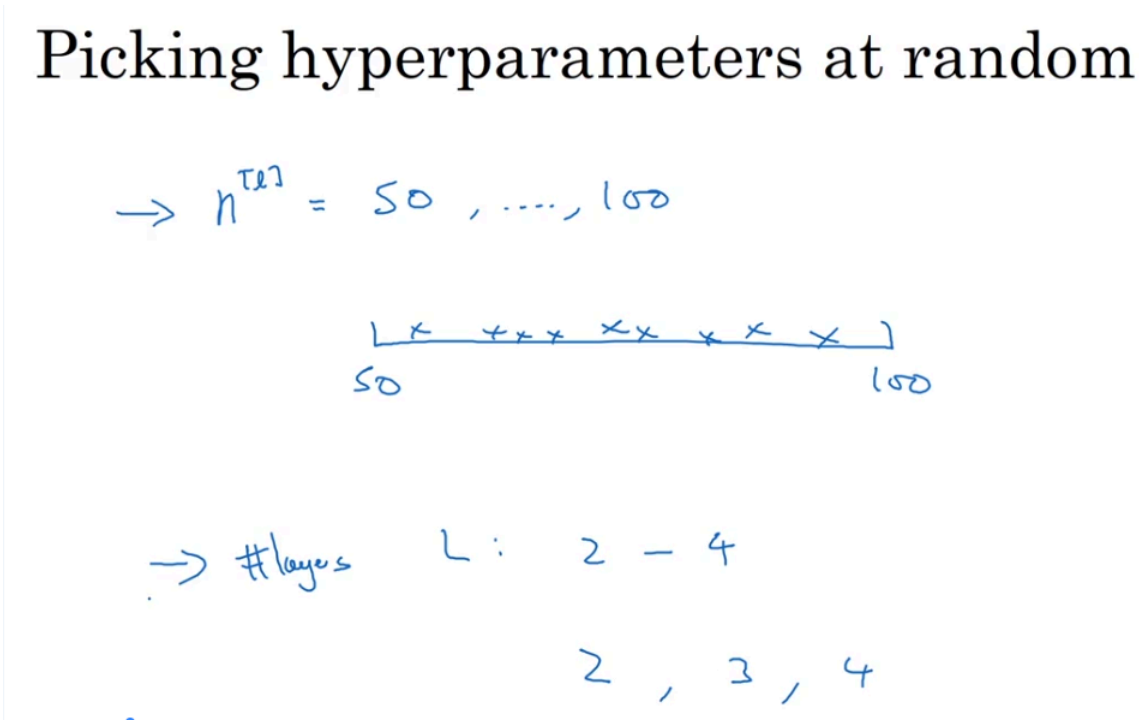
So let's say in this two-dimensional example that you sample these points, and maybe you found that this point work the best and maybe a few other points around it tended to work really well, then in the course of the final scheme what you might do is zoom in to a smaller region of the hyperparameters, and then sample more density within this space.

Or maybe again at random, but to then focus more resources on searching within this blue square if you're suspecting that the best setting, the hyperparameters, may be in this region. So after doing a coarse sample of this entire square, that tells you to then focus on a smaller square. You can then sample more densely into smaller square. So this type of a coarse to fine search is also frequently used.

And by trying out these different values of the hyperparameters you can then pick whatever value allows you to do best on your training set objective, or does best on your development set, or whatever you're trying to optimize in your hyperparameter search process.

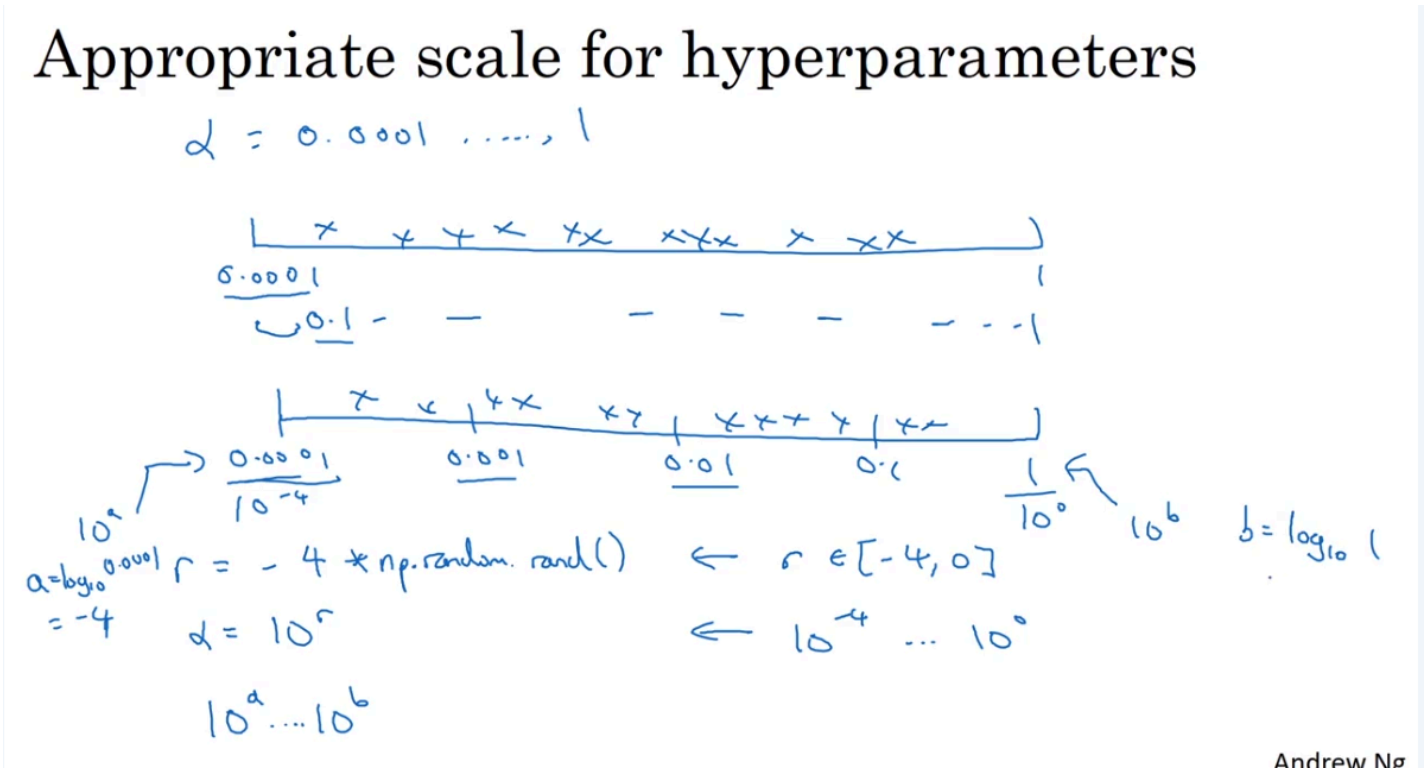
Using an Appropriate Scale to pick Hyperparameters

Let's say that you're trying to choose the number of hidden units, n[l], for a given layer l. And let's say that you think a good range of values is somewhere from 50 to 100. In that case, if you look at the number line from 50 to 100, maybe picking some number values at random within this number line.



There's a pretty visible way to search for this particular hyperparameter. Or if you're trying to decide on the number of layers in your neural network, we're calling that capital L. Maybe you think the total number of layers should be somewhere between 2 to 4. Then sampling uniformly at random, along 2, 3 and 4, might be reasonable. Or even using a grid search, where you explicitly evaluate the values 2, 3 and 4 might be reasonable. So these were a couple examples where sampling uniformly at random over the range you're contemplating; might be a reasonable thing to do. But this is not true for all hyperparameters.

How to choose a good learning rate?



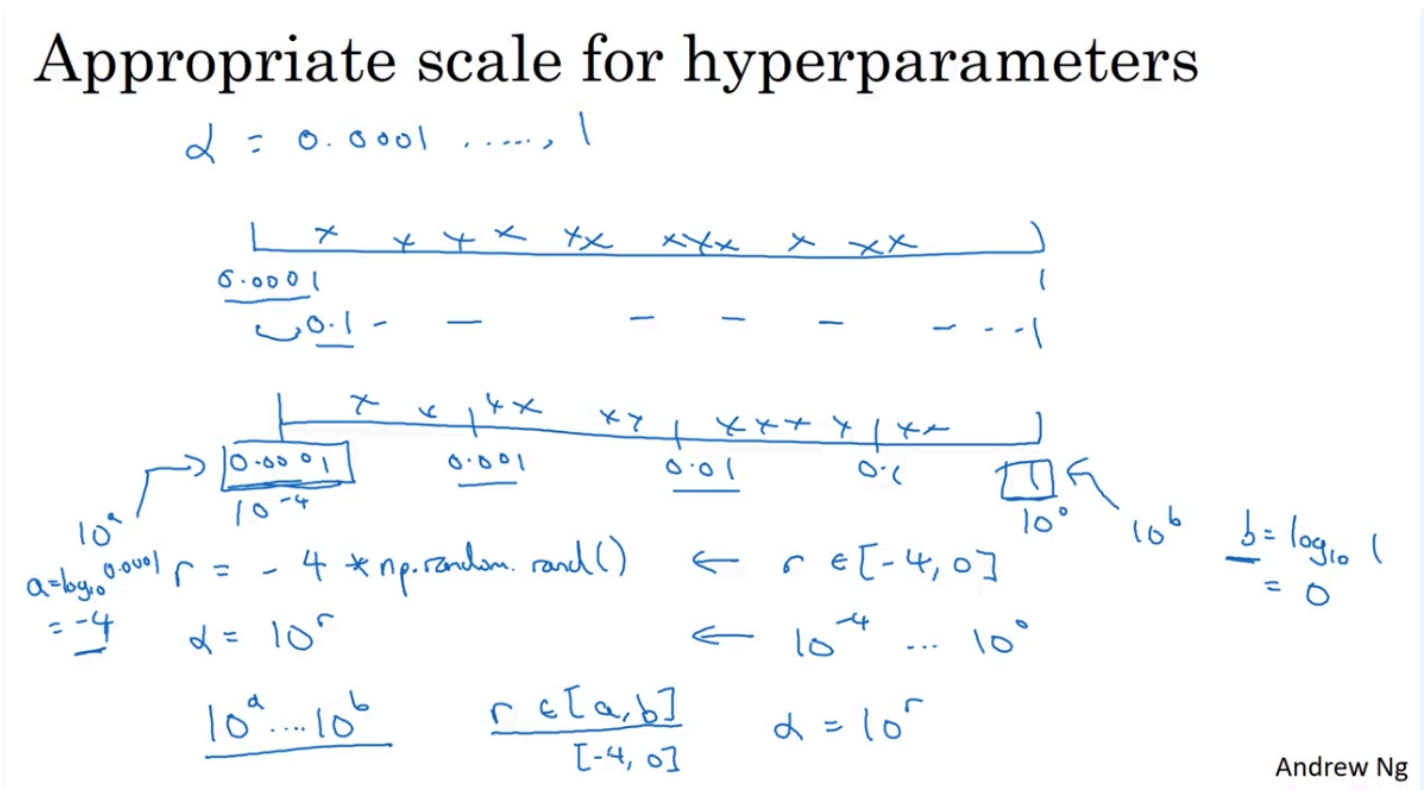
what if we wanna choose a learning rate in range 0.0001 to 1, shall we use linear line?

The answer is NO!

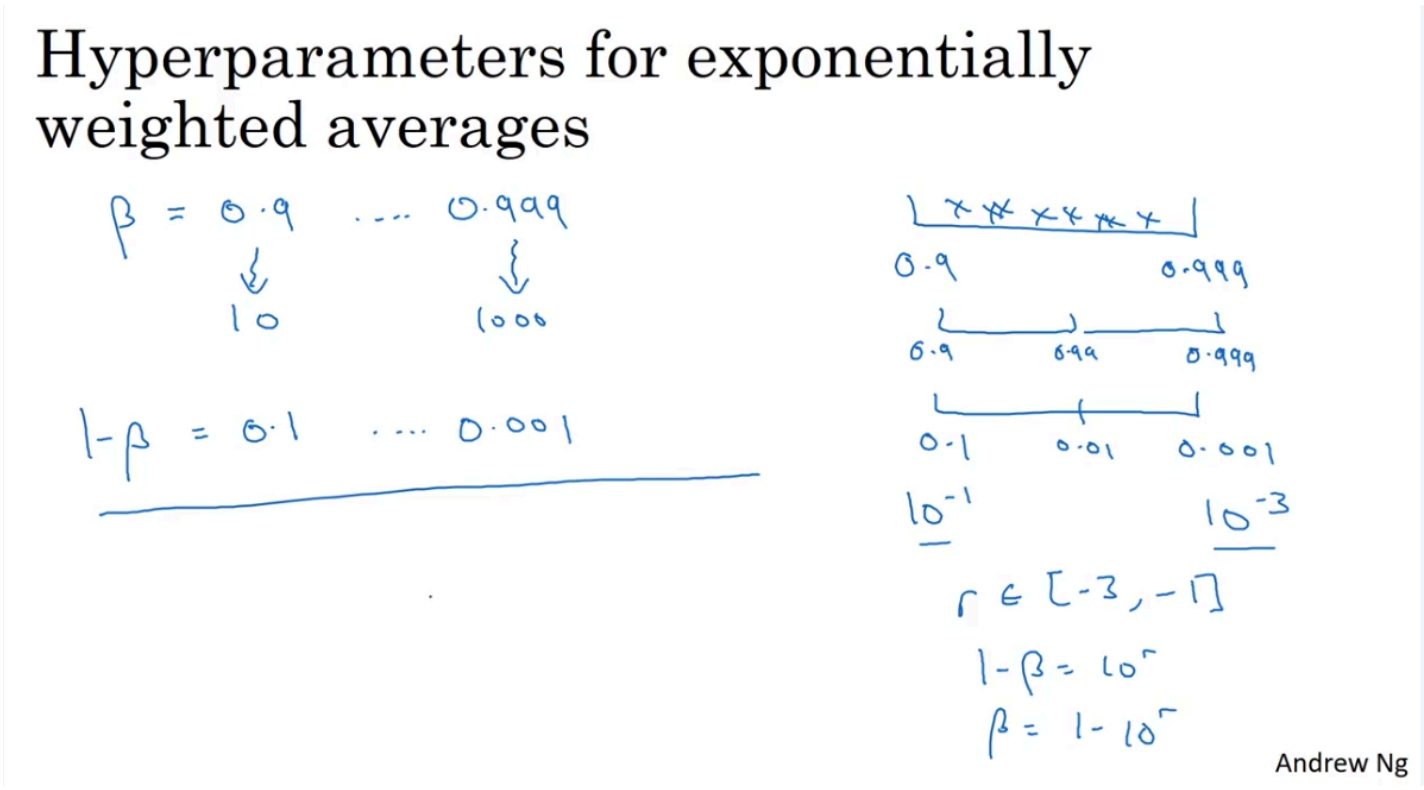
→ use logarithmic scale instead (like the second line)

Now you have more resources dedicated to searching between 0.0001 and 0.001, and between 0.001 and 0.01, and so on. So in Python, the way you implement this, is let **r = -4 * np.random.rand().**

And then a randomly chosen value of alpha, would be alpha = 10^r.



How to choose a nice appropriate beta?



same as above, but instead of finding beta, we could find 1 - beta,

let: r in range (-3, -1)

let: 1 - beta = 10^ r

⇒ beta = 1 - 10^r

So if beta goes from 0.9 to 0.9005, it's no big deal, this is hardly any change in your results.

But if beta goes from 0.999 to 0.9995, this will have a huge impact on exactly what your algorithm is doing, right? In both of these cases, it's averaging over roughly 10 values. But here it's gone from an exponentially weighted average over about the last 1,000 examples, to now, the last 2,000 examples.

And it's because that formula we have, 1 / 1- beta, this is very sensitive to small changes in beta, when beta is close to 1. So what this whole sampling process does, is it causes you to sample more densely in the region of when beta is close to 1.

So what this whole sampling process does, is it causes you to sample more densely in the region of when beta is close to 1. Or, alternatively, when 1- beta is close to 0. So that you can be more efficient in terms of how you distribute the samples, to explore the space of possible outcomes more efficiently.

Hyperparameters Tuning in Practice: Pandas vs. Caviar

You have now heard a lot about how to search for good hyperparameters. Before wrapping up our discussion on hyperparameter search, I want to share with you just a couple of final tips and tricks for how to organize your hyperparameter search process.

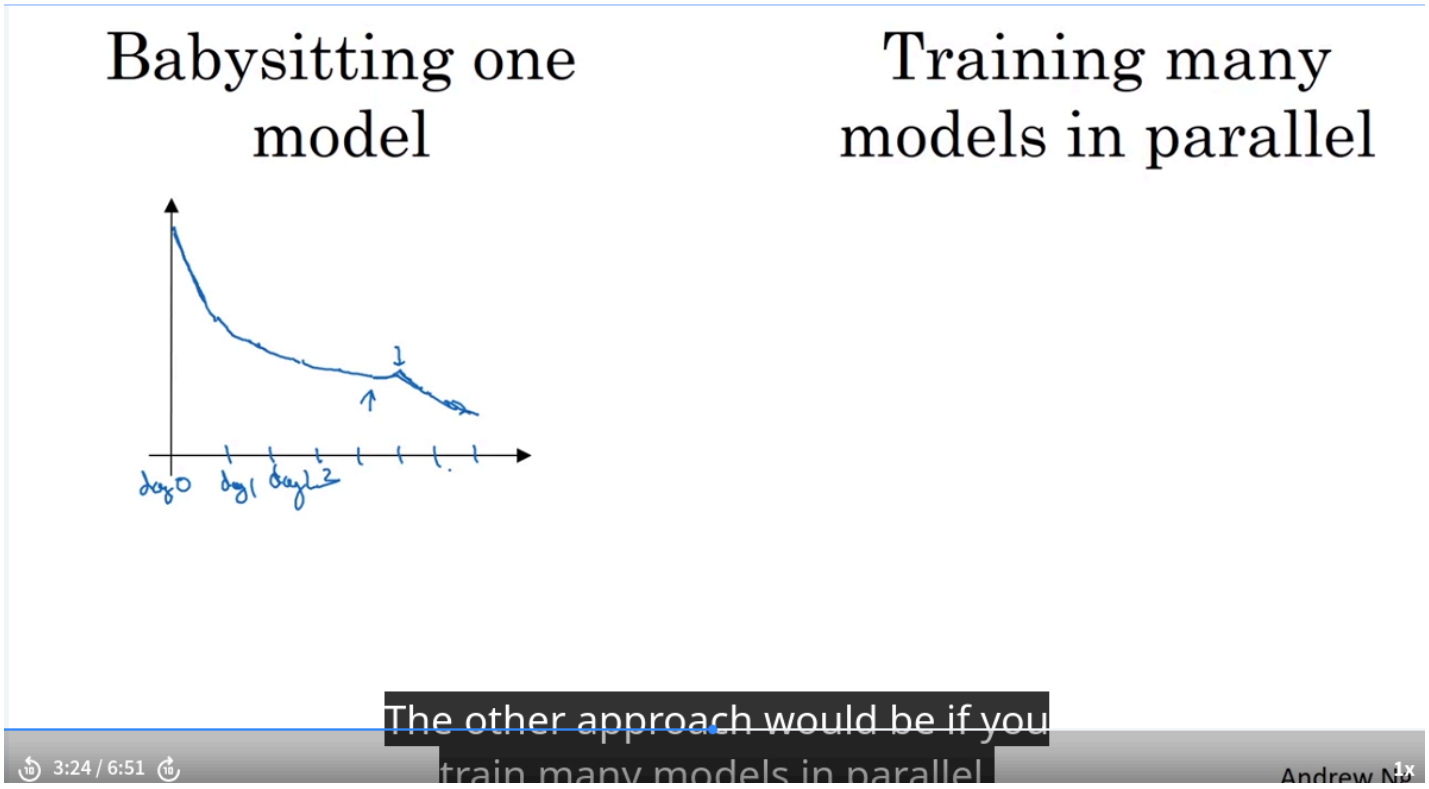
If we dont have really huge GPUs, and could only train a one model at a time

So, for example, on

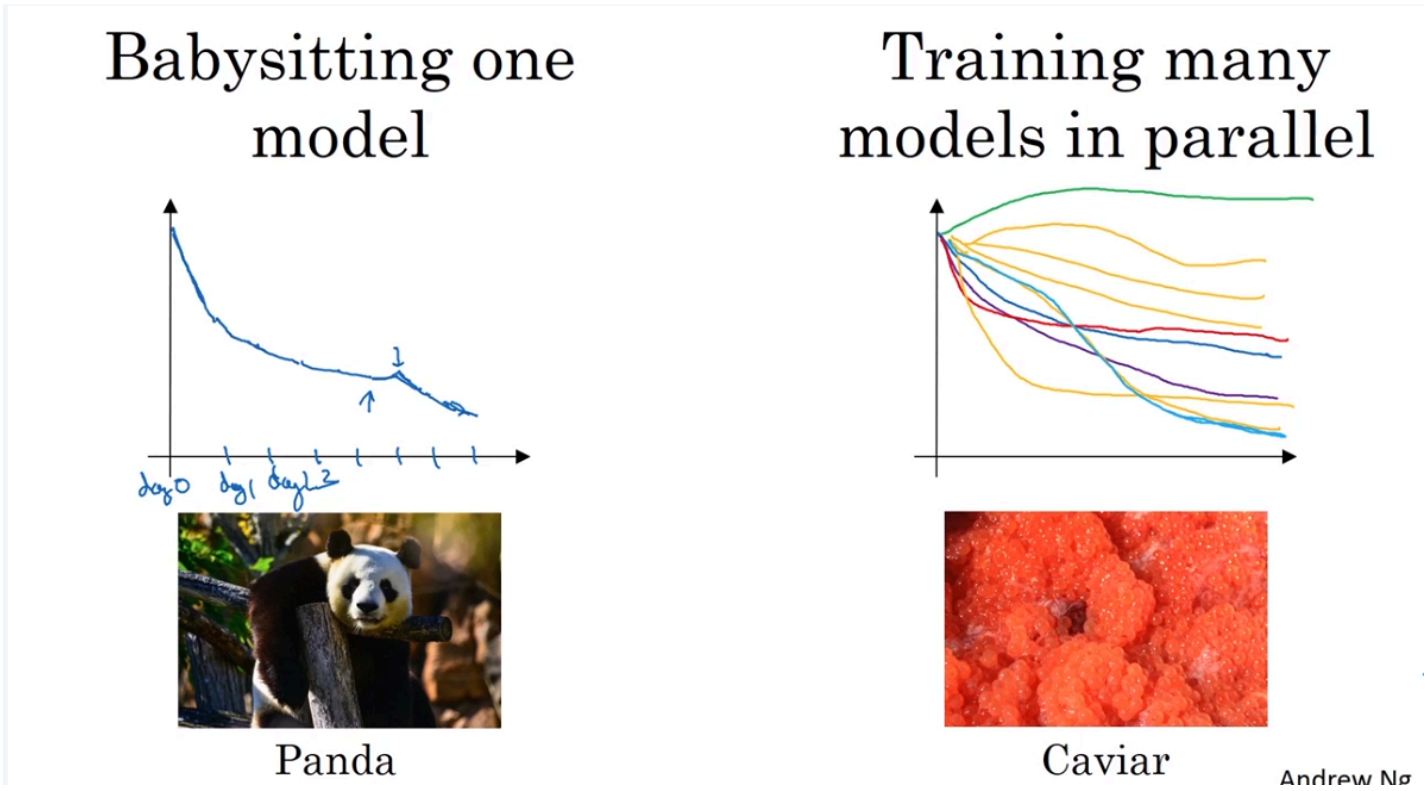
Day 0 you might initialize your parameter as random and then start training. And you gradually watch your learning curve, maybe the cost function J or your dataset error or something else, gradually decrease over the first day. Then at the end of day one, you might say, gee, looks it's learning quite well, I'm going to try increasing the learning rate a little bit and see how it does. And then maybe it does better.

And then that's your Day 2 performance. And after two days you say, okay, it's still doing quite well. Maybe I'll fill the momentum term a bit or decrease the learning variable a bit now, and then you're now into Day 3.

And every day you kind of look at it and try nudging up and down your parameters. And maybe on one day you found your learning rate was too big. So you might go back to the previous day's model, and so on. But you're kind of babysitting the model one day at a time even as it's training over a course of many days or over the course of several different weeks. So that's one approach, and people that babysit one model, that is watching performance and patiently nudging the learning rate up or down. But that's usually what happens if you don't have enough computational capacity to train a lot of models at the same time.



The other approach would be if you train many models in parallel.



So you might have some setting of the hyperparameters and just let it run by itself ,either for a day or even for multiple days, and then you get some learning curve like that; and this could be a plot of the cost function J or cost of your training error or cost of your dataset error, but some metric in your tracking.

And then at the same time you might start up a different model with a different setting of the hyperparameters. And so, your second model might generate a different learning curve, maybe one that looks like that. I will say that one looks better. And at the same time, you might train a third model, which might generate a learning curve that looks like that, and another one that, maybe this one diverges so it looks like that, and so on. Or you might train many different models in parallel, where these orange lines are different models, right, and so this way you can try a lot of different hyperparameter settings and then just maybe quickly at the end pick the one that works best. Looks like in this example it was, maybe this curve that look best.

So to make an analogy, I'm going to call the approach on the left the panda approach. When pandas have children, they have very few children, usually one child at a time, and then they really put a lot of effort into making sure that the baby panda survives. So that's really babysitting. One model or one baby panda. Whereas the approach on the right is more like what fish do. I'm going to call this the caviar strategy. There's some fish that lay over 100 million eggs in one mating season.